Indirect Access

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 29/08/2025

Learning Journey and Evidence

Prior Knowledge and Experience

Before attempting this task, I had already built a strong foundation through both my previous studies and the earlier tasks (P1–P7).

From my A-level studies, I am well versed in passing parameters by references since I had to rely on the feature pretty much for all my A-level programs. That said, I had never heard or used const(Constant) references before as A-level programs were somewhat small programs(less than 400 lines of code) so they didn't really focus on efficiency.

As for pointers, I am very familiar with them as I had to use them for all my programmes done for levels that included bubble sort, insertion sort, queue, stacked, linked list and binary tree. From this, I know that pointers are really useful to manipulate data but it can be very risky, as pointing to the wrong “thing” can and will, in some cases, crash the program.

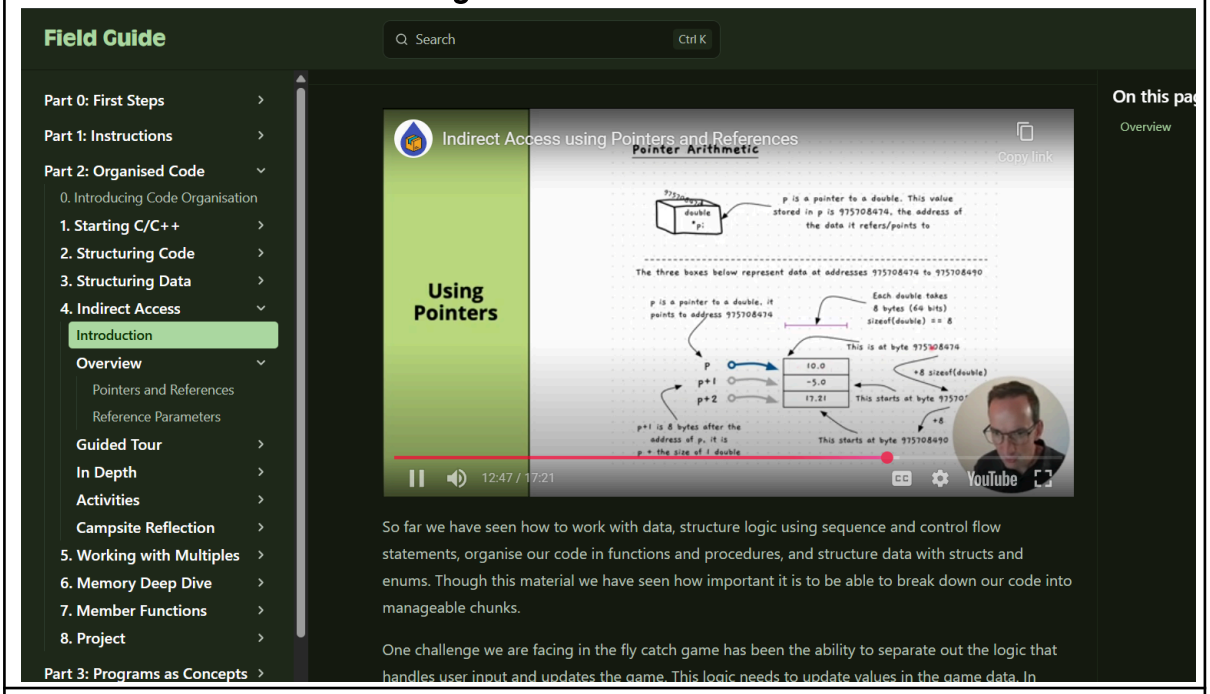
Through P1–P7, I further developed my programming knowledge step by step:

- P1 (Learning Plan): Helped me set clear goals, organize my study approach, and identify areas where I needed to improve.
- P2 (Building and Running): Gave me practical skills of executing programs, as well as debugging common errors.
- P3 (Sequence and Data): Strengthened my understanding of variables, operators, and data handling, which is the basis of problem-solving experience.
- P4 (Control Flow): Taught me how to use conditionals and loops effectively to control the execution path of programs.
- P5 (C/C++): Reinforced syntax rules and highlighted the importance of precision in coding (e.g., C++ being case-sensitive).
- P6 (Structuring Code): Showed me how to modularize programs using functions and procedures, improving readability and reusability.
- P7 (Structuring Data): Introduced enums, structures, and the organization of data in memory, preparing me for more advanced concepts like pointers and references.

Together, these tasks gave me both the theoretical knowledge and practical experience to approach indirect access with confidence. I can now see how pointers and references build on earlier concepts such as arrays, control flow, and structured data, and how they enable more modular and efficient program design.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

27/08/2025 - Read the field guide extensively
Watch the included in the field guide: 
From reading and watching the explanation video, I feel confident tackling this topic.

28/08/2025 - Coding[Field Guide → Part 2 → 4.Indirect access → Guided Tour → Explore References]

Account database

For this example, lets walk through creating a simple account database. This database can keep track of usernames and their associated system role.

The first step, as always, should be to think about the entities we will need within this program. We can start in this case with users and roles. Have a think about how you would represent these, and the coding structures you would use for this.

Make sure your user has at least a username, password, and permission level. Have roles: no access, user, admin, and super-user.

Once you have a plan, have a go at creating this in a new project file. We can make use of the utilities we created earlier to make it easier to read and write values via the terminal.

My code;

explore_references > references.cpp > ...

```cpp
#include "splashkit.h"
#include "utilities.h"

/** emumarated day type created to allows only specific roles
 * role allowed are: NO_ACCESS, USER, ADMIN, SUPER_USER
 */
enum role_level
{
    NO_ACCESS,
    USER,
    ADMIN,
    SUPER_USER
};

/**structured data type user_data, it stores data of the user:
 * these data are: username, password, role
 */
struct user_data
{
    string username;
    string password;
    role_level role;
};
```

```
/** function that read the option for role from user input and return the role
 *
 * @return role of the user
 */
role_level read_role()
{
    int role_input = read_integer_limit("Enter role level (0 for NONE, 1 for USER 2 For ADMIN, 3 for SUPER_USER): ", 0, 3);

    switch (role_input)
    {
    case 0:
        return NO_ACCESS;
        break;
    case 1:
        return USER;
        break;
    case 2:
        return ADMIN;
        break;
    case 3:
        return SUPER_USER;
        break;
    default:
        write_line("Invalid role level. Setting role to NONE");
        return NO_ACCESS;
        break;
    }
}
```

```
/** Function to return the role in string data type from role_level data type
 * @param role of role_level data type
 *
 * @return role in string data type
 */
string to_string(role_level role)
{
    switch (role)
    {
    case NO_ACCESS:
        return "No Access";
        break;

    case USER:
        return "User";
        break;

    case ADMIN:
        return "Admin";
        break;

    case SUPER_USER:
        return "Super User";
        break;

    default:
        return "ERROR!!!";
        break;
    }
}
```

```
/** Prints the details of the user
 *
 * @param user of data type user_data; passed as a const reference to save memory
 */
void print_user(const user_data &user)
{
    write_line("The username is " + user.username + " and their role is " + to_string(user.role));
}
```

```
/** update user any details, can be one edit or multiple edits
 *
 * @param user of data type user_data; passed by reference
 */
void update_user(user_data &user)
{
    int choice;
    string line;
    string flag;

    do
    {
        // menu
        write_line("");
        write_line("1. Update username");
        write_line("2. Update password");
        write_line("3. Update role");
        choice = read_integer_limit("Enter choice: ", 1, 3);

        switch (choice)
        {
        case 1:
            write_line();
            write("Enter new username: ");
            user.username = read_line();
            write_line("Username change successfully.");
            break;
        case 2:
            write_line();
            write("Enter old password: ");
            line = read_line();
```

```
            if (user.password == line)
            {
                line = "";
                write("Enter new password: ");
                line = read_line();
                user.password = line;
                write_line("Password change successfully.");
            }
            else
            {
                write_line("Incorrect Password");
            }
            break;
        case 3:
            write_line();
            write("Enter new role: ");
            user.role = read_role();
            write_line("Role change successfully.");
            break;

        default:
            write_line("ERROR!!!");
            break;
        }
        write_line("");
        write("Anything more to change[y/n]: ");
        flag = read_line();

    } while (flag == "y" || flag == "Y");
}
```

```
int main()
{
    user_data user = {"My_user_name", "monash", USER};

    update_user(user);
    print_user(user);

    return 0;
}
```

i understand how to use utilities as well as deepen my understanding of passing by references

28/08/2025 - Coding[Field Guide → Part 2 → 4.Indirect access → Guided Tour → Fly catch]

```
#include "splashkit.h"

/**
 * The spider has a location on the screen.
 *
 * @field x the distance from the left side of the window
 * @field y the distance from the top of the window
*/
struct spider_data
{
    double x;
    double y;
};

/**
 * The fly can appear and escape (disappear), but is otherwise
 * located on the screen at a certain point.
 *
 * @field appeared  indicates if the fly is shown
 * @field x its distance from the left of the window (to the center)
 * @field y its distance from the top of the window (to the center)
 * @field appear_at_time  the time when the fly will appear
 * @field escape_at_time  the time when the fly will escape
*/
struct fly_data
{
    bool appeared;
    double x;
    double y;
    int appear_at_time;
    int escape_at_time;
};
```

```
/**
 * The game involves a spider and a fly. The player is
 * controlling the spider and the fly is trying to escape.
 *
 * @field spider  the data about the spider
 * @field fly     the data for the fly
 */
struct game_data
{
    spider_data spider;
    fly_data fly;
};


const string GAME_TIMER = "GameTimer";
const int GAME_SCREEN_WIDTH = 800;
const int SCREEN_HEIGHT = 600;
const int GAME_SCREEN_HEIGHT = SCREEN_HEIGHT - 100;
const int SPIDER_RADIUS = 25;
const int SPIDER_SPEED = 3;
const int FLY_RADIUS = 10;
```

```
/**
 * Draw the spider on the screen
 *
 * @param spider  the spider's data
 */
void draw_spider(spider_data spider)
{
    fill_circle(color_black(), spider.x, spider.y, SPIDER_RADIUS);
}

/** procedure to handle inputs from the user
 *
 * @param spider of data type spider_data
 */
void handle_input(spider_data &spider)
{
// Spider movement code
    if ((key_down(RIGHT_KEY)) && (spider.x + SPIDER_RADIUS < GAME_SCREEN_WIDTH)) // spider move right
    {
        spider.x += SPIDER_SPEED;
    }
    if ((key_down(LEFT_KEY)) && (spider.x - SPIDER_RADIUS > 0)) // spider move left
    {
        spider.x -= SPIDER_SPEED;
    }

    if ((key_down(DOWN_KEY)) && (spider.y + SPIDER_RADIUS < SCREEN_HEIGHT)) // spider move down
    {
        spider.y += SPIDER_SPEED;
    }
    if ((key_down(UP_KEY)) && (spider.y - SPIDER_RADIUS > 100)) // spider move up
    {
        spider.y -= SPIDER_SPEED;
    }
    // spider movenent code end
}
```

```
/** function to determine if the fly has to appeared
 *
 * @param fly of data type fly_data passed by constant reference; contains fly details
 * @param current_time of data type long pass by constant reference; conatins current time of the program
 *
 * @return bool value for if the fly has appeared.
 */
bool time_to_appear(const fly_data &fly,const long &current_time)
{
  return !fly.appeared && current_time > fly.appear_at_time;
}

/** function to determine if the fly has to escape
 *
 * @param fly of data type fly_data passed by constant reference; contains fly details
 * @param current_time of data type long pass by constant reference; conatins current time of the program
 *
 * @return bool value for if the fly has appeared.
 */
bool time_to_escape(const fly_data &fly, long current_time)
{
  return fly.appeared && current_time > fly.escape_at_time;
}

```

```
/** procedure to update fly details
 *
 * @param fly of data type fly_data; store data about the fly
 * @param current_time of data type long pass by constant reference; conatins current time of the program
 * @param countgame of data type int passed by reference; counts the number of games
 *
 */

void update_fly(fly_data &fly, long current_time,int &countgame)
{
  // Check if the fly should appear
  if (time_to_appear(fly, current_time))
  {
    // Make the fly appear
    fly.appeared = true;

    // Give it a new random position
    fly.x = rnd(GAME_SCREEN_WIDTH);
    fly.y = 100+ rnd(GAME_SCREEN_HEIGHT);

    // Set its escape time
    fly.escape_at_time = current_time + 2000 + rnd(5000);
    countgame += 1;

  }
  else if (time_to_escape(fly, current_time))
  {
    fly.appeared = false;
    fly.appear_at_time = current_time + 1000 + rnd(2000);
  }
}
```

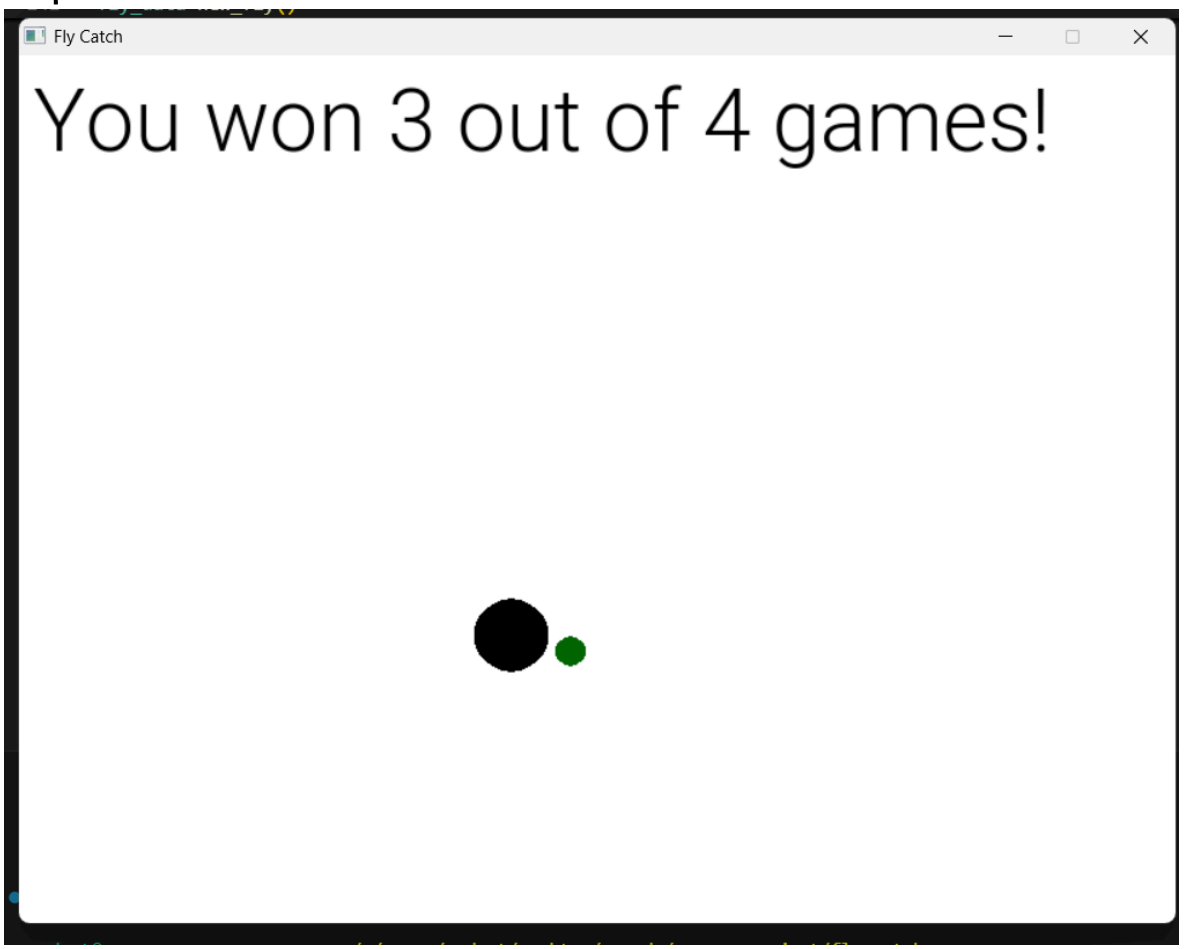
```cpp
/**
 * Draw the fly if it has appeared
 *
 * @param fly the fly data
*/
void draw_fly(fly_data fly)
{
    if (fly.appeared)
    {

        fill_circle(color_dark_green(), fly.x, fly.y, FLY_RADIUS);
    }
}

/** function to check if the fly has been caught
 *
 * @param spider of data type spider_data passed by constant reference
 * @param fly of data type fly_data passed by constant reference
 *
 * @return a boolean value for if the fly has been caught of not
 */
bool spider_caught_fly(const spider_data &spider, const fly_data &fly)
{

  return fly.appeared && circles_intersect(
    spider.x, spider.y, SPIDER_RADIUS,
    fly.x, fly.y, FLY_RADIUS);
}
```

```cpp
/** Upadte the game screen
 * @param game of data type game_data passed by reference; conatin the game data
 * @param current_time of data type long passed by reference; contain the current game time
 * @param countwin of data type int passed by reference; counts the numeber od game won
 * @param countgame of data type int passed by reference; counts the number of games
 *
 * @return a boolean value for if the fly has been caught of not
 */
void update_game(game_data &game, long current_time, int &countwin,int &countgame)
{
  update_fly(game.fly, current_time,countgame);

  if ( spider_caught_fly(game.spider, game.fly))
  {
    countwin += 1;
    game.fly.appeared = false;
    game.fly.appear_at_time = current_time + 1000 + rnd(2000);
  }
}


/**
 * Draw the spider and the fly to the screen.
 *
 * @param game the game data
*/
void draw_game(game_data game, int countwin, int &countgame)
{
    clear_screen(color_white());
    draw_spider(game.spider);
    // generate the game counter on screen
    draw_text("You won " + std::to_string(countwin) + " out of " + std::to_string(countgame) + " games!", color_black(), "main", 60, 10, 10);

    draw_fly(game.fly);
    refresh_screen(60);
}
```

```
/**
 * Initialise a new spider and return it to the caller.
 * The spider will be in the center of the screen.
 *
 * @return an initialised spider
*/
spider_data new_spider()
{
    spider_data spider;

    spider.x = GAME_SCREEN_WIDTH / 2;
    spider.y = GAME_SCREEN_HEIGHT / 2;
    return spider;
}

/**
 * Initialise and return new fly data. This will not have appeared,
 * but will be at a random position, and will appear within 1
 * to 3 seconds.
 *
 * @return the newly initialised fly
*/
fly_data new_fly()
{
    fly_data fly;

    fly.x = rnd(GAME_SCREEN_WIDTH);
    fly.y = rnd(GAME_SCREEN_HEIGHT);

    fly.appeared = false;
    fly.appear_at_time = 1000 + rnd(2000);
    fly.escape_at_time = 0;

    return fly;
}
```

```
int main()
{
    int countgame = 0;
    int countwin = 0;
    game_data game;

    game.spider = new_spider();
    game.fly = new_fly();

    load_font("main", "Roboto-Light.ttf");                    // Load front for text on screen
    open_window("Fly Catch", GAME_SCREEN_WIDTH, SCREEN_HEIGHT); // create the screen

    create_timer(GAME_TIMER); // create a timer for the game to based off of.
    start_timer(GAME_TIMER);

    while (!quit_requested())
    {
        handle_input(game.spider);
        update_game(game, timer_ticks(GAME_TIMER),countwin,countgame);

        draw_game(game, countwin, countgame);

        process_events();
    }
}
```

Output:



I understood how to implement parameters passed by reference for my previous program.

28/08/2025 - Coding[Field Guide → Part 2 → 4.Indirect access → Guided Tour → Maze game]

Description:

Maze Game

Text based adventure games have a player move through a number of virtual rooms within the program. This code requires us to set up rooms and paths that move between them.

If you think about the game you will see that we need pointers to implement this. Each path needs to know the room it goes to. This cannot be a copy of the room - as circular paths would become impossible.

Instead, what we need is to have one place where the rooms exist, and have the paths point to their destination room.

The player's location can then also be a pointer to a room. You can think of the value as saying "this is the room I am in" and it points to the room

code:

maze_game > field_maze.cpp > ...

```cpp
#include <string>

using std::string;

typedef struct room_data *room_ptr;

struct path_data
{
  string    description;
  room_ptr  destination;
};

struct room_data
{
  string title;
  string description;
};

/**
 * New path populates the data in a path struct and returns it.
 *
 * @param description the description of the path
 * @param destination the destination of the path - a pointer to the room it goes to
 * @return path_data with the indicated details
 */
path_data new_path(string description, room_ptr destination)
{
  path_data result = { description, destination };

  return result;
}
```

```
/**
 * New room populates the data in a room struct and returns it.
 *
 * @param title         The title of the room
 * @param description   The description of the room
 * @returns             The new room data
 */
room_data new_room(string title, string description)
{
  room_data result = { title, description };

  return result;
}

/**
 * Outputs the path's description to the terminal. Showing the index of the path
 * and the description, but no details of where it goes.
 *
 * @param idx   the index of the path - used to select it from the list
 * @param path  the path to print (const reference)
 */
void print_path(int idx, const path_data &path)
{
  printf("%d - %s\n", idx, path.description.c_str());
}
```

```
/**
 * Output the room's title and description to the terminal.
 *
 * @param room a pointer to the room to print
 */
void print_room(room_ptr room)
{
  printf("%s\n-----\n%s\n", room->title.c_str(), room->description.c_str());
}

/**
 * Move the player through the selected path, setting the player's current room
 * to the destination of the path.
 *
 * @param current_room a reference to the player's current room (a pointer to the room_data)
 * @param path a constant reference to the path to move through
 */
void move_player(room_ptr &current_room, const path_data &path)
{
  current_room = path.destination;
}
```

```c

/**
 * Reads an integer value from the user. If the user enters a non-integer value,
 * the function will continue to prompt the user until they enter a valid value.
 *
 * @param prompt the prompt message to display to the user
 * @return int the integer value entered by the user
 */
int read_integer(const string &prompt)
{
  int result = 0;

  printf("%s", prompt.c_str());
  while (scanf(" %d", &result) != 1) // Read value, and try to convert to int
  {
    // Convert failed, as input was not a number
    scanf("%*[^\n]"); // Read past the end of the current line
    printf("Please enter a whole number.\n");
    printf("%s", prompt.c_str());
  }

  return result;
}
```

```c
int main()
{
  room_data r1 = new_room("Room 1", "You are in a happy place");
  room_data r2 = new_room("Room 2", "This is room 2");
  room_data r3 = new_room("Room 3", "This is room 3");

  path_data p1 = new_path("A large sliding door", &r2);
  path_data p2 = new_path("An open corridor", &r3);

  room_ptr current_room = &r1;

  print_room(current_room);
  print_path(0, p1);
  print_path(0, p2);

  int option = read_integer("Option: ");

  while(option < 0 || option > 1)
  {
    printf("Choose a value between 0 and 1\n");
    option = read_integer("Option: ");
  }

  if ( option == 0 )
  {
    move_player(current_room, p1);
  }
  else
  {
    move_player(current_room, p2);
  }

  print_room(current_room);
}
```

Output:

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/maze_game
$ ./maze1
Room 1, You are in a happy place
0. A large sliding door
1. An open corridor
Option: 0
Room 2, This is room 2

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/maze_game
$ ./maze1
Room 1, You are in a happy place
0. A large sliding door
1. An open corridor
Option: 1
Room 3, This is room 3

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/maze_game
$
```

I struggled a lot with this program, as I had to remember/relearned how to work with pointers.
I was able to do it in the end and it is working correctly.

29/08/2025 - Coding[Field Guide → Part 2 → 4.Indirect access → Test yourself → Bank system]

Output template:

```
Simple transaction record!

Enter account name: Andrew
Enter interest percent (+/-): 5.1
Enter initial amount: 1000

Actions:
1: Print account details
2: Deposit
3: Withdraw
4: Add interest
5: Quit
Option: 2

Amount deposited: -10
Please enter a value larger than or equal to 0
Amount deposited: 10
Balance: 1010.000000

Actions:
1: Print account details
2: Deposit
3: Withdraw
4: Add interest
5: Quit
Option: 3

Amount withdrawn: -10
Please enter a value larger than or equal to 0
Amount withdrawn: 10
Balance: 1000.000000

Actions:
1: Print account details
2: Deposit
3: Withdraw
4: Add interest
5: Quit
Option: 4

Interest for how many days: 30
Rate is 5.1% PA = 0.4191780822%
Interest = 4.1917808219
Balance = 1004.1917808219

Actions:
1: Print account details
2: Deposit
3: Withdraw
4: Add interest
5: Quit
Option: 1

Account: Andrew
Rate: 5.1%
Balance: 1004.1917808219

Actions:
1: Print account details
2: Deposit
3: Withdraw
4: Add interest
5: Quit
Option: 5
```

My code:

```
#include "splashkit.h"
#include <stdio.h>
/** Struct data type to store account details;
 * name of account holder
 * Annual Interest percentage
 * Amount in account
 */
struct account_details
{
    string name;
    double intrest;
    double amount;
};

/** function to validate user input for intrest amount
 *
 * @param prompt conatin the output message to let the user know what to enter; pass as const reference
 *
 * @return intrest of type double
 */
double read_interest(const string &prompt)
{
    write(prompt);
    string line = read_line();
    double num;
    while (!is_double(line))
    {
        write_line("Please enter a numerical value greater than or equal to 0.");
        write(prompt);
        line = read_line();
    }
    num = stod(line);
    return num;
}
```

```

/** function to validate user input for amount
 *
 * @param prompt conatin the output message to let the user know what to enter; pass as const reference
 *
 * @return amount of type double
 */
double read_amount(const string &prompt)
{
    write(prompt);
    string line = read_line();
    bool flag = false;
    double num;
    do
    {
        while (!is_double(line))
        {
            write_line("Please enter a numerical value greater than or equal to 0.");
            write(prompt);
            line = read_line();
        }
        num = stod(line);
        if (num >= 0.0)
        {
            flag = true;
        }
        else
        {
            write_line("Please enter a value >= 0.");
            write(prompt);
            line = read_line();
        }
    } while (!flag);
    return num;
}
```

```
/** function to validate user input for choice
 * @param prompt conatin the output message to let the user know what to enter; pass as const reference
 * @param lower_limit contain the smallest valid number that can be entered by then user
 * @param upper_limit contain the largest valid number that can be entered by then user
 *
 *
 * @return choice of type integer
 */
int read_integer_limit(string prompt, int lower_limit, int upper_limit)
{
    write(prompt);
    string line = read_line();
    bool flag = false;
    int num;
    do
    {
        while (!is_integer(line))
        {
            write_line("Please enter a whole number between " + std::to_string(lower_limit) + " and " + std::to_string(upper_limit));
            write(prompt);
            line = read_line();
        }
        num = stoi(line);
        if (num >= lower_limit && num <= upper_limit)
        {
            flag = true;
        }
        else
        {
            write_line("Please enter a whole number between " + std::to_string(lower_limit) + " and " + std::to_string(upper_limit));
            write(prompt);
            line = read_line();
        }
    } while (!flag);
    return num;
}
```

```
/** Procedure to output the user account details
 *
 * @param account of data type account_details; contain the deatils of the account; passed as const reference
 *
 */
void print_acc_details(const account_details &account)
{
    printf("Account: %s\nRate: %lf\nBalance: %lf\n", account.name.c_str(), account.intrest, account.amount);
}

/** Procedure to allow the user to deposit money into the account
 *
 * @param amount  of data type double; contain the amount in account; passed as reference as this value will change
 *
 */
void deposit(double &amount)
{
    amount = amount + read_amount("Amount deposited: ");
    printf("Amount = %lf\n", amount);
}
```

```
/** Procedure to allow the user to withdraw money into the account
 *
 * @param amount  of data type double; contain the amount in account; passed as reference as this value will change
 *
 */
void withdrawn(double &amount)
{
    double w = read_amount("Enter amount to withdraw: ");
    if (w > amount)
    {
        write_line("Insufficient funds!");
    }
    else
    {
        amount -= w;
    }
}

/** Procedure to allow the user to process intrest into the account
 *
 * @param intrest of data type double; contain the intrest percentage per annum of the account; passed as const reference
 * @param amount  of data type double; contain the amount in account; passed as reference as this value will change
 */
void add_intrest(const double &interest, double &amount)
{
    int days;
    double current_interest;
    days = read_integer_limit("Intrest for how many days: ", 1, 365);
    current_interest = interest * days / 365;
    printf("Rate is %lf%%  PA = %lf%%\n", interest, current_interest);
    printf("Interest = %lf\n", amount * current_interest / 100);
    amount += amount * current_interest / 100;
    printf("Amount = %lf\n", amount);
}
```

```
/** Menu function to process access the different option of the program
 *
 * @param account of data type account_details; contain the deatils of the account; passed by reference
 *
 * @return a boolean value for if the user want to quit the program
 */
bool menu(account_details &account)
{
    int choice;
    write_line("");
    write_line("Actions:");
    write_line("1. Print account details");
    write_line("2. Deposit");
    write_line("3. Withdraw");
    write_line("4. Add interest");
    write_line("5. Quits");
    choice = read_integer_limit("option ", 1, 5);

    switch (choice)
    {
    case 1:
        write_line("");
        print_acc_details(account);
        break;
    case 2:
        write_line("");
        deposit(account.amount);
        break;
    case 3:
        write_line("");
        withdrawn(account.amount);
        break;
```

```
        case 3:
            write_line("");
            withdrawn(account.amount);
            break;
        case 4:
            write_line("");
            add_intrest(account.intrest, account.amount);
            break;
        case 5:
            return true;
            break;

        default:
            write_line("");
            write_line("Error!!!!");
            break;
        }
        return false;
    }
```

```

int main()
{
    printf("Simple transaction record!\n\n");

    account_details account;
    string choice;

    printf("Enter account name: ");
    account.name = read_line();
    account.intrest = read_interest("Enter interest percent (-/+): ");
    account.amount = read_amount("Enter initial ammount: ");

    do
    {
    } while (!menu(account));  // continue to process the user request until the user menu return true
}
```

Output:

```
Vedant@LAPTOP-Q7B9365O MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/bank_system
$ ./b
Simple transaction record!

Enter account name: Andrew
Enter interest percent (-/+): 5.1
Enter initial ammount: 1000

Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
5. Quits
option -10
Please enter a whole number between 1 and 5
option 2

Amount deposited: -10
Please enter a value >= 0.
Amount deposited: 10
Amount = 1010.000000

Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
5. Quits
option 3

Enter amount to withdraw: 5000
Insufficient funds!
```

```
Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
5. Quits
option 3

Enter amount to withdraw: 10

Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
5. Quits
option 1

Account: Andrew
Rate: 5.100000
Balance: 1000.000000

Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
5. Quits
option 4

Intrest for how many days: 30
Rate is 5.100000%  PA = 0.419178%
Interest = 4.191781
Amount = 1004.191781
```

```
Actions:
1. Print account details
2. Deposit
3. Withdraw
4. Add interest
esktop\mycode\my_c++_project\fly_catch_revisited
Option 5

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/bank_system
$
```

The bank system task helped me see how references can be used in programs that require consistent updates across multiple functions. For example, account balances could be updated through references without needing additional return values. This gave me a clearer view of how references reduce redundancy and improve collaboration between different modules in a program.

30/08/2025 - Coding[Field Guide → Part 2 → 4.Indirect access → Test Yourself → Reaction Game]

Reaction game

Create a small target clicking game that works on reaction time to scale difficulty. When the game starts, fill a circle on the screen in a random location. The objective is to click this as quickly as possible.

When the target is clicked, move it to a new position and adjust the radius to be proportional to the time taken. If the user takes more than 1 second (1000ms) make the circle slowly grow larger.

Show the number of targets hit, and the number of milliseconds of the fastest reaction - this could look similar to the example below:

My code:

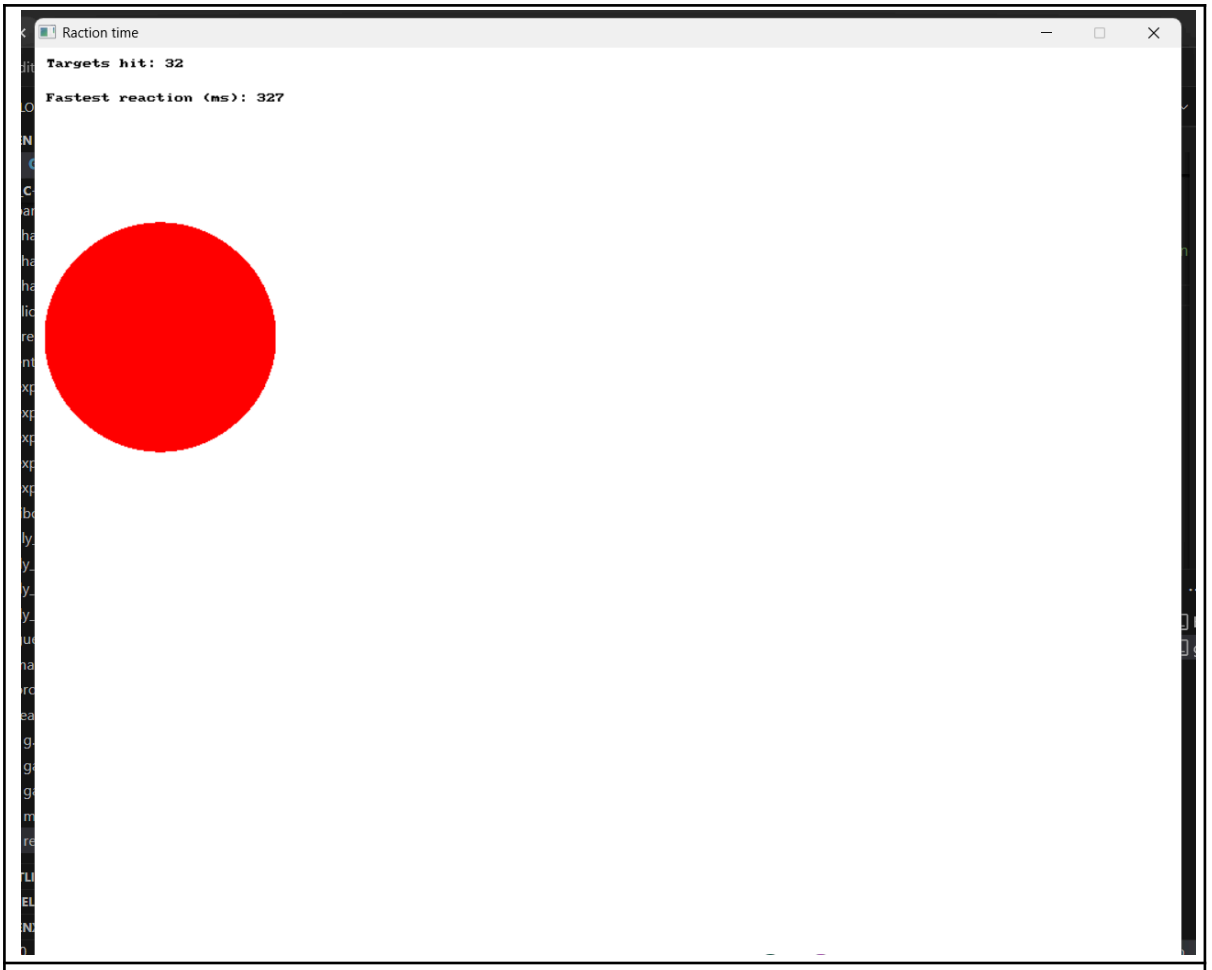
```c
#include "splashkit.h"
#include <stdio.h>

//game timer
const string GAME_TIMER = "gametimer";

//screen size
const double SCREEN_WIDTH = 1000;
const double SCREEN_HEIGHT = 800;

//target radius
const int INITIAL_RADIUS = 50;

/** structured data type that contain the game details
 *
 * Game details:
 * -target of data type circle
 * -time of data time int that store the best reaction time
 * -score of data type int that store the number of target hit
 */
struct game_data
{
    circle target;
    int time;
    int score = 0;
};
```

```c
/** Procedure that handle the user input and update the target data when hit
 *
 * @param game of data type game_data that contain data about the game: passed by reference
 */
void handle_input(game_data &game)
{
    do
    {
        process_events();
    } while ((!point_in_circle(mouse_position(), game.target)) && !quit_requested()); // Wait for user to hit target or if the user quits game

    int temp;
    temp = timer_ticks(GAME_TIMER);
    if (temp < game.time)                    //compare and store reaction if it is faster taht the stored one
    {
        game.time = timer_ticks(GAME_TIMER);
    }
    game.score += 1;                        //increament  game score
    reset_timer(GAME_TIMER);
    game.target.center = random_screen_point();
    temp = temp / 1000 * 100;               //Calculate the radius based on reaction 100ms per second
    if (temp < 5)                           //If radius equates to less than 5, set radius to 10
    {
        game.target.radius = 10;
    }
    else
    {
        game.target.radius = temp;
    }

}
```

```
/** Procedure that the update version of the screen
 *
 * @param game of data type game_data that contain data about the game: passed as const reference
 */
void update_game(const game_data &game)
{
    clear_screen(color_white());
    fill_circle(color_red(), game.target);
    draw_text("Targets hit: " + std::to_string(game.score), color_black(), 10, 10);
    draw_text("Fastest reaction (ms): " + std::to_string(game.time), color_black(), 10, 40);
    refresh_screen();
}

/** Procedure that draw the screen
 *
 * @param game of data type game_data that contain data about the game: passed as reference
 */
void draw_game(game_data &game)
{

    clear_screen(color_white());
    while (!quit_requested())
    {
        process_events();
        update_game(game);
        handle_input(game);
    }
}
```

```

int main()
{
    game_data game;

    create_timer(GAME_TIMER);
    start_timer(GAME_TIMER);
    game.time = 1000000;
    open_window("Raction time", SCREEN_WIDTH, SCREEN_HEIGHT);

    game.target.center = random_screen_point();
    game.target.radius = INITIAL_RADIUS;

    draw_game(game);
}
```

Output:



This exercise consolidated my understanding of references in an interactive setting. I used references to manage variables that tracked timing and user responses. The main takeaway was how references simplify code, avoid unnecessary copying, and make real-time updates more efficient. Completing this task gave me confidence to use references naturally in my own programs.

Brief Summary of Concepts

Concept / Commands	Key Idea / Concept
Pointer	A variable that stores the memory address of another variable. Used for direct memory access and dynamic memory management.
Reference	An alias for an existing variable. It doesn't create a new copy but provides another name for the same memory location.
Pass by value	A copy of the variable is passed to a function. Changes inside the function do not affect the original variable.
Pass by reference	The actual variable (via pointer or reference) is passed to a function. Changes inside the function affect the original variable.

Pointer and Reference

```
#include "splashkit.h"

void test_proc(int *p1, int &p2)
{
    *p1 = 10;
    p2 = 20;
}

int main()
{
    // Example usage of std::format
    int number = 42;

    int *ptr;
    int &ref = number;

    ptr = &number;

    test_proc(&number, number);

    return 0;
}
```

	Response
1: Describe number.	It is a variable of data type integer which as been assigned value 42
2: Describe ptr.	Declaration of a pointer
3: Explain what this does	• Declaration of a reference called *ref* that reference *number* • *ref* is just another name (alias) for *number*, not a new variable. • Any change to *ref* also changes *number*.
4: Explain what this does.	ptr now stores the address of *number*. Using **ptr*, you can access or modify *number*.
5: What occurs here?	Calls the function *test_proc*. First argument: &number → passes the address of *number* (so inside p1, it points to *number*). Second argument: number → passed by reference to p2, so inside the function, p2 is an alias for number.
6: Describe p1	Inside test_proc(int *p1, int &p2), p1 is a pointer. It points to *number* (because we passed &number).

7: Describe p2	Inside test_proc, p2 is a reference. It is another name (alias) for *number*. Any assignment to p2 directly changes *number*.
8: Explain what happens here - in relation to the call indicated by arrow 5.	Dereferences p1 and assigns 10. Since p1 points to *number*, this sets *number* = 10.
9: Explain what happens here - in relation to the call indicated by arrow 5.	Assigns 20 to p2. Because p2 is a reference to *number*, this changes number = 20. This overwrites the earlier 10 from step 8. After the function ends, *number* remains 20 in main.

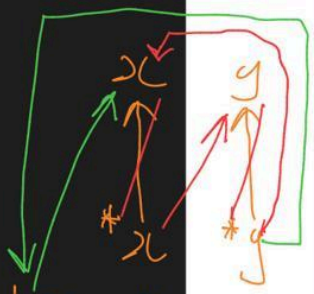
Describe swap

```c
void swap_1(int x, int y) //local swap using a temporary variable
{
    int temp = x;
    x = y;
    y = temp;

    printf("In swap peocedure: X = %d, Y = %d\n",x,y);
}

void swap_2(int *x, int *y) //global Swap using pointers
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

void swap_3(int &x, int &y) //global swap  using references
{
    int temp = x;
    x = y;
    y = temp;
}

int main()
{
    int x = 5;
    int y = 10;

    swap_1(x,y);
    printf("\nswap 1: X = %d, Y = %d\n",x,y);

    swap_2(&x,&y);  //references of x and y are passed since it wouldn't work other wise
    printf("\nswap 2: X = %d, Y = %d\n",x,y);

    swap_3(x,y);
    printf("\nswap 3: X = %d, Y = %d",x,y);
}
```

```
$ clang++ swap.cpp -o s -l splashkit

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/swap
$ ./s
In swap peocedure: X = 10, Y = 5

swap 1: X = 5, Y = 10

swap 2: X = 10, Y = 5

swap 3: X = 5, Y = 10
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/swap
```

temp = x	x = y	y = temp
∴ temp ← x	x ← y	y ← temp
temp ← 5	x ← 10	y ← 5
∴ temp has value 5	x has value 10	y has value 5

```c
void swap_1(int x, int y) //local swap using a temporary variable
{
    int temp = x;
    x = y;
    y = temp;

    printf("In swap peocedure: X = %d, Y = %d\n",x,y);
}

void swap_2(int *x, int *y) //global Swap using pointers
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

void swap_3(int &x, int &y) //global swap  using references
{
    int temp = x;
    x = y;
    y = temp;
}

int main()
{
    int x = 5;
    int y = 10;

    swap_1(x,y);
    printf("\nswap 1: X = %d, Y = %d\n",x,y);

    swap_2(&x,&y);  //references of x and y are passed since it wouldn't work other wise
    printf("\nswap 2: X = %d, Y = %d\n",x,y);

    swap_3(x,y);
    printf("\nswap 3: X = %d, Y = %d",x,y);
}
```



temp = *x

∴ temp = &x

*x points to x
*y points to y

① temp take the value address that *x points to that is temp take the value address of variable x.

② *x now point to the value address being pointed by *y. therefore *y now points to y

③ *y now point to variable x since temp was storing address of variable x.

&x references x
&y references y

∴ these are just alias for the original variable

∴ since swapping them (the alias), the original variable will also be swap.

Reflection

What is the most important thing you learned from this and why?

The most important thing I learned is how references allow data to be accessed and modified indirectly, without needing to duplicate it. This is powerful because it improves efficiency (no unnecessary copying), makes it easier to work with large or complex data structures, and allows functions to interact with the same underlying variable instead of working on a copy.
Understanding this is key to writing cleaner and more effective code, especially as programs become larger and more complex.

How do you see references helping you modularise larger programs?

References make it possible to pass data between functions and modules without copying everything. Instead of each function working in isolation with its own copy, references let functions collaborate on the same piece of data. This makes modules more reusable and reduces redundancy, since different parts of the program can operate on shared structures consistently.
For example, a function can update a variable passed by reference, and those changes are immediately visible everywhere else in the program. This modularity makes it easier to separate concerns and keep the program organized.

Many modern languages make extensive use of references, but do not include an equivalent of pointers. Why do you think this may be?

Modern languages emphasize safety and readability. Pointers are powerful but risky; they allow direct memory manipulation, which can easily cause errors like memory leaks, dangling pointers, or buffer overflows.

References, on the other hand, provide much of the same flexibility but in a safer, more controlled way. They let you share and modify data without exposing raw memory addresses. This makes programs more secure, less error-prone, and easier to maintain, which aligns with the design goals of modern high-level languages.

References

Generative AI acknowledgement

No AI was used for either the code or this document.